

Lekce 11: Vzor Page Object Model (POM) v Cypressu

Cíle

- Implementovat vzor Page Object Model (POM) pro zvýšení udržitelnosti testů.
- Zapouzdřit prvky stránky a akce do znovupoužitelných modulů.
- Opakovaně používat page objekty v rámci více testů, aby se minimalizovala duplicita a zlepšila škálovatelnost.

Rozpis obsahu

A. Pochopení návrhových vzorů POM

Co je návrhový vzor a proč je důležitý?

Definice:

Návrhový vzor je obecné, znovupoužitelné řešení běžného problému, který se vyskytuje při návrhu softwaru. Nejedná se o hotový návrh, který lze přímo převést do kódu, ale spíše o šablonu či plán, který lze aplikovat v mnoha různých situacích k řešení návrhové výzvy.

Proč jsou návrhové vzory důležité:

1. Znovupoužitelná řešení:

- Poskytují osvědčené postupy pro řešení běžných problémů, takže není nutné „znovu vynalézat kolo“.

2. Zlepšená komunikace:

- Návrhové vzory nabízejí vývojářům společný slovník. Když někdo zmíní „Factory pattern“ nebo „Observer pattern“, tým okamžitě chápe obecný přístup.

3. Vyšší udržitelnost:

- Díky známé struktuře je kód snáze pochopitelný, udržitelný a rozšiřitelný, protože se drží osvědčených postupů.

4. Lepší škálovatelnost:

- Vzory pomáhají organizovat kód tak, aby byl škálovatelný a dokázal lépe reagovat na budoucí změny či nové funkce.

5. Oddělení zodpovědností:

- Podporují oddělení odpovědností, čímž snižují provázanost a zvyšují přehlednost jednotlivých komponent.

6. Rychlejší vývoj:

- Díky předpřipravené šabloně pro řešení problémů se vývojáři mohou soustředit na konkrétní implementaci místo na celkovou strukturu.

Page Object Model

1. Co je Page Object Model?

- **Definice:**

Page Object Model (POM) je návrhový vzor v testovací automatizaci, který odděluje reprezentaci stránky (její prvky a akce) od samotné testovací logiky. Každá stránka aplikace je reprezentována třídou či modulem obsahujícím selektory (lokátory) a metody (akce) pro interakci se stránkou.

- **Účel:**

- Oddělit detaily UI od testovacího kódu.
- Zlepšit udržitelnost a čitelnost testů.

2. Výhody použití POM v testovací automatizaci:

- **Udržitelnost:**

Při změně UI je potřeba upravit pouze page objekt, nikoliv všechny testy.

- **Znovupoužitelnost:**

Běžné akce (např. přihlášení, vyplnění formuláře) jsou zapouzdřeny na jednom místě a lze je použít ve více testech.

- **Čitelnost:**

Testy jsou přehlednější a soustředí se na logiku testu, protože nízkourovňové interakce jsou abstrahovány do page objektů.

- **Škálovatelnost:**

Umožňuje budování modulárního testovacího balíku, který roste společně s aplikací.

B. Implementace POM

1. Vytvoření tříd/modulů pro stránky:

- Každá stránka je reprezentována vlastním JavaScriptovým souborem (nebo třídou), který obsahuje selektory a metody pro danou stránku.
- **Příklad struktury adresářů:**

```
cypress/  
├── e2e/  
│   ├── login.spec.js  
│   └── registration.spec.js  
└── pages/  
    ├── LoginPage.js  
    └── RegistrationPage.js
```

2. Zapouzdření prvků stránky a akcí:

- **Příklad: LoginPage.js**

```
// cypress/support/pages/LoginPage.js  
  
class LoginPage {  
  // Definice selektorů  
  
  usernameInput () => cy.get('[data-cy="login-username-input"]'),  
  passwordInput () => cy.get('[data-cy="login-password-input"]'),  
  submitButton () => cy.get('[data-cy="login-submit-button"]'),  
  errorMessage () => cy.get('[data-cy="error-invalid"]'),  
  successMessage () => cy.get('[data-cy="login-success-message"]')  
  
  // Definice akcí  
  login(username, password) {  
    this.usernameInput().clear().type(username);  
    this.passwordInput().clear().type(password);  
    this.submitButton().click();  
  }  
}  
  
export const loginPage = new LoginPage();
```

- **Příklad: RegistrationPage.js**

```
// cypress/support/pages/RegistrationPage.js
class RegistrationPage {
  usernameInput () => cy.get('[data-cy="reg-username-input"]'),
  emailInput () => cy.get('[data-cy="reg-email-input"]'),
  passwordInput () => cy.get('[data-cy="reg-password-input"]'),
  submitButton () => cy.get('[data-cy="reg-submit-button"]'),
  successMessage () => cy.get('[data-cy="reg-success-message"]'),
  errorMessage () => cy.get('[data-cy="reg-error-message"]')

  register(user) {
    this.elements.usernameInput().clear().type(user.username);
    this.elements.emailInput().clear().type(user.email);
    this.elements.passwordInput().clear().type(user.password);
    this.elements.submitButton().click();
  }
}

export const registrationPage = new RegistrationPage();
```

3. Znovupoužití page objektů napříč testy:

- Po definici lze page objekty importovat do testovacích souborů a provádět akce bez opakování selektorů.
- **Příklad v testu:**

```
/// <reference types="cypress" />
import { loginPage } from '../support/pages/LoginPage';

describe('User Login @login', () => {
  beforeEach(() => {
    cy.visit('login.html');
  });

  it('should log in successfully with valid credentials', () => {
    loginPage.login('demoUser', 'demoPass');
    loginPage.elements.successMessage().should('be.visible');
  });

  it('should display an error with invalid credentials', () => {
    loginPage.login('wrongUser', 'wrongPass');
    loginPage.elements.errorMessage().should('be.visible');
  });
});
```

Implementace POM s parametrizovaným konstruktorem

V některých aplikacích můžete mít na stránce více podobných widgetů (např. karty produktů, dashboardové widgety), které sdílejí stejnou funkcionalitu, ale liší se ID nebo jiným atributem. K ošetření těchto případů lze v POM použít parametrizovaný konstruktor.

Příklad: Widget Page Object

Představme si několik widgetů na dashboardu, z nichž každý zobrazuje informace o jiném produktu. Každý widget má unikátní ID a chcete pracovat s konkrétním widgetem.

HTML příklad (widget):

```
<div class="widget" data-testid="widget1">
  <h2 data-cy="widget-title">Widget Title</h2>
  <p data-cy="widget-description">This is a description.</p>
  <button data-cy="widget-action">Action</button>
</div>
```

Parametrizovaný Page Object:

```
// cypress/support/pages/WidgetPage.js

class WidgetPage {
  constructor(widgetId) {
    // Kontejner widgetu je vybrán na základě předaného widgetId
    this.widgetSelector = `[data-test="${widgetId}"]`;
  }

  // Vrátí kontejner widgetu
  getContainer() {
    return cy.get(this.widgetSelector);
  }

  // Vrátí titulek widgetu
  getTitle() {
    return this.getContainer().find('[data-cy="widget-title"]');
  }

  // Vrátí popis widgetu
  getDescription() {
    return this.getContainer().find('[data-cy="widget-description"]');
  }

  // Vrátí akční tlačítko ve widgetu
  getActionButton() {
    return this.getContainer().find('[data-cy="widget-action"]');
  }

  // Akce: klikne na akční tlačítko widgetu
  clickActionButton() {
    this.getActionButton().click();
  }
}

// Export tovární funkce pro vytvoření nového widgetu dle widgetId
export const getWidget = (widgetId) => new WidgetPage(widgetId);
```

Využití v testu:

```
/// <reference types="cypress" />
import { getWidget } from '../support/pages/WidgetPage';

describe('Dashboard Widgets', () => {
  beforeEach(() => {
    cy.visit('dashboard.html');
  });

  describe('Widget Interactions', () => {
    it('should display correct title and description for widget1', () => {
      const widget = getWidget('widget1');
      widget.getTitle().should('contain', 'Widget Title');
      widget.getDescription().should('contain', 'This is a description.');
```

});

it('should perform an action when widget1 action button is clicked', () => {

const widget = getWidget('widget1');

widget.clickActionButton();

// Přidejte zde aserce související s akcí

});

});

});

Další příklad – nákupní košík

```
export default class CartItem {
  private index: number = 0
  constructor(index: number = 0) {
    // logika konstruktora
    this.index = index
  }

  product = () => cy.get(`[data-testid="product-${this.index}"]`)

  productTitle = () => this.product().find('[data-testid="product-header"]')
  productType = () => this.product().find('[data-testid="product-type-attribute"]').eq(0)
  editProductBtn = () => this.product().find('[data-testid*= "header-detail-button"]')
  detailProduct = () => this.product().find('[data-testid="button-detail-product"]')

  openProductDetail = () => {
    this.editProductBtn().click()
  }
}
```

Využití v testu

```
it('Should open product detail from the cart on Product page', () => {

  // ... kód

  let cartItem = new CartItem(0)
  cartItem.productTitle().should('contain', 'Pivo s rumom')
  cartItem.openProductDetail()

  // ... kód
})
```

C. Osvědčené postupy pro POM

1. Udržujte page objekty čisté a zaměřené:

- Každý page objekt by měl obsahovat pouze selektory a akce specifické pro danou stránku.
- Nemíchejte nesouvisející logiku.

2. Vyvarujte se duplicity:

- Opakovaně používejte selektory a akce prostřednictvím page objektů.
- Sdílejte-li více stránek společné prvky či akce, zvažte vytvoření základní (base) třídy.

3. Zajistěte škálovatelnost:

- Navrhujte page objekty tak, aby byly snadno rozšiřitelné společně s aplikací.
- Používejte jasná, popisná jména metod, která odrážejí uživatelské akce (např. `login()`, `register()`, `fillForm()`).

4. Udržitelnost:

- Při změně UI upravte pouze příslušný page objekt.
- Testovací soubory udržujte přehledné a zaměřené na aserce, zatímco interakce řeší page objekty.

5. Používejte popisné konvence pojmenování:

- **describe() bloky:** používejte název funkce nebo stránky (např. „Testy přihlašovací stránky“).
- **it() bloky:** dodržujte vzor AAA (Arrange-Act-Assert), např. „mělo by se úspěšně přihlásit se správnými údaji“.

D. Použití vlastních příkazů Cypress společně s POM

- Zatímco page objekty kapslírují interakce specifické pro stránku, vlastní příkazy (custom commands) mohou kapslírovat často opakované akce napříč stránkami.
- **Příklad:** Jestliže je přihlášení běžnou akcí používanou v mnoha testech, definujte vlastní příkaz, který volá `loginPage.login()` :

```
// cypress/support/commands.js
import { loginPage } from './pages/LoginPage';
Cypress.Commands.add('login', (username, password) => {
  loginPage.login(username, password);
});
// Využití v testu:
cy.login('demoUser', 'demoPass');
```

E. Struktura testů v projektech Cypress

- **Organizace adresářů:**
 - **fixtures:** ukládají externí data (např. JSON).
 - **integration/e2e:** testovací specifikace uspořádané podle funkcionalit.

- **plugins:** konfigurace nebo rozšíření chování Cypress.
- **support:** vlastní příkazy a soubory page objektů.
- **Konzistence:**
 - Dodržujte jednotné pojmenování a strukturu souborů napříč projektem.
- **Škálovatelnost:**
 - Organizujte testy do logických skupin pomocí vnořených `describe()` bloků.
 - Používejte vlastní příkazy a page objekty ke snížení duplicity a usnadnění údržby.

Aktivita

- **Vytvoření testu pro ukázkovou aplikaci:**
 - Prozkoumejte aplikaci a navrhnete testy.

Zdroje

- **Dokumentace Cypress k POM:**
[Page Object Model in Cypress](#)
- **Ukázkové projekty:**
Prozkoumejte open-source projekty na GitHubu, které implementují vzor POM s Cypress.